Lab Handout 3-D

Packet Error Rate and End-to-End Latency Measurement

Capstone — Module 3: System Integration and Data Collection

1 Objective

Define and measure the system's two primary reliability and timing metrics: packet error rate (Packet Error Rate (PER)) and end-to-end latency. By the end of this handout you will have a quantified PER decomposed by pipeline stage and a latency budget verified on hardware for at least two spreading factor settings.

2 Prerequisites

- Lab Handout 3-C complete (first live frame captured; 10-frame field comparison passed).
- System running stably: cooperative-target ping-pong (Target A DATA + Target B ACK) producing intercepted traffic, Threat forwarding frames over Universal Asynchronous Receiver/Transmitter (UART), Field-Programmable Gate Array (FPGA) producing `frame_valid` pulses with both DATA (Beacon ID `0x01`) and ACK (Beacon ID `0x02`) rows visible in the ILA.

3 Background: Performance Metrics

Packet Error Rate

PER captures the fraction of transmitted radio packets (DATA or ACK; PAUSE packets are excluded from this baseline because the Threat half-duplex shadows its own TX) that fail to produce a valid feature vector at the FPGA output. It accumulates losses from the Radio Frequency (RF) link, the UART transport, and CRC verification:

$$\text{PER} = 1 - \frac{N_{\text{valid features}}}{N_{\text{transmitted packets}}}$$

A single aggregate PER is useful for system characterization, but the more important quantity is the *decomposition* by stage. The decomposition reveals which part of the pipeline is losing packets.

End-to-End Latency

End-to-end latency is the time from radio packet TX start (by an originating cooperative target) to feature vector availability in the FPGA BRAM:

$$L_{\text{e2e}} = L_{\text{RF}} + L_{\text{RX_FW}} + L_{\text{UART}} + L_{\text{FPGA}}$$

At 115200 baud, the UART contribution is fixed: $43 \times 10/115{,}200 = 3.73$ ms. The FPGA pipeline contribution is negligible (<0.1 ms at 100 MHz). The RF on-air time dominates and depends strongly on the LoRa spreading factor.

4 Procedure

4.1 Step 1 — PER baseline (single station, 5-minute run)

1. Power off Target B and disable the Threat's button. Run with Target A only, configured at SF7, +14 dBm, 2-second fixed initiator interval. Without Target B there are no ACK rows; the dataset will only contain `pkt_type = 0` (DATA) entries from Beacon ID `0x01`.

2. Map the per-station frame count register in the FPGA to an output (the Module 2 LH2-F feature BRAM stores this as bytes 8–11 of the feature vector for each Beacon ID). Alternatively, add a simple 32-bit counter gated on `frame_valid` AND `beacon_id = 0x01` and route it to the ILA.

3. Record Target A's sequence number from its USART2 debug terminal at the start of the run: N_{start}.

4. Run for exactly 5 minutes (300 seconds). Record Target A's sequence number at the end: N_{end}. Total transmitted: $N_{\text{tx}} = N_{\text{end}} - N_{\text{start}}$.

5. Read the FPGA frame count for Beacon ID `0x01` at the end of the run: N_{valid}.

6. Compute: PER $= 1 - N_{\text{valid}}/N_{\text{tx}}$.

4.2 Step 2 — PER decomposition

A single PER number does not reveal where packets are lost. Decompose it by stage:

Loss Source	How to Measure	Notes
RF loss	$N_{\text{tx}} - N_{\text{rx_log}}$	Compare Target A sequence numbers to Threat USART2 log; no RTL changes needed
CRC loss	`frame_reject` counter	Add a 16-bit saturating counter in RTL (see below); read via ILA after the run
FIFO overflow	`fifo_full` on LED3 / ILA	Already routed; note whether it ever asserted during the run

1. **Add a saturating counter for `frame_reject`.** RF loss is measured from sequence numbers and `fifo_full` is already observable on LED3 and the ILA — neither requires new RTL. Only `frame_reject` needs a counter to produce a number at the end of a run. Add the following to `ingest_top.vhd`, with `flag` wired to `frame_reject`, and route `cnt` to an ILA probe:

```
-- Saturating counter for frame_reject
process(CLK100MHZ)
begin
  if rising_edge(CLK100MHZ) then
```

```
      if rst_n = '0' then
        cnt <= (others => '0');
      elsif frame_reject = '1' and cnt /= x"FFFF" then
        cnt <= cnt + 1;
      end if;
    end if;
  end process;
```

2. After the 5-minute run, read the `frame_reject` counter via the ILA and note whether `fifo_full` (LED3) ever asserted.

3. Compute the per-stage loss as a fraction of N_{tx}. Enter in the table:

Stage	**Packets Lost**	**Fraction of N_{tx}**
RF (not received by Threat)		
CRC rejects (`frame_reject` counter)		
FIFO overflow (`fifo_full` asserted?)	Y / N	
Total (aggregate PER)		

Interpreting PER Decomposition

The decomposition reveals which domain "owns" the packet loss:

- **RF-dominated:** link budget or environment issue (Module 1 domain). Increasing TX power or reducing distance will help.
- **FIFO-overflow-dominated:** baud rate drift causing runaway byte generation, or parser back-pressure under burst traffic. Check the STM32 clock configuration and the VHSIC Hardware Description Language (VHDL) `uart_rx` baud divisor.
- **CRC-dominated:** firmware/RTL mismatch not fully resolved in LH3-C. A persistent non-zero CRC reject rate after LH3-C should be investigated further before data collection.
- **FIFO-dominated:** the pipeline cannot drain fast enough under burst traffic. This is unlikely with a single station but may appear in the cooperative-target ping-pong + PAUSE-burst stress test (LH3-E).

4.3 Step 3 — GPIO toggle wiring for latency measurement

The General-Purpose Input/Output (GPIO) toggle method provides a true end-to-end latency measurement without relying on synchronized clocks between boards.

1. In Target A's firmware (LH1-F), confirm the existing TX-trigger GPIO pulse on `TX_TRIG` (`PB10`) added in LH1-F Step 6 fires immediately before each DATA TX. If you removed it, re-add the line: `HAL_GPIO_WritePin(TX_TRIG_GPIO_Port, TX_TRIG_Pin, GPIO_PIN_SET)` just before commanding TX mode.
2. In `ingest_top.vhd`, route `frame_valid` to a Nexys A7 PMOD output pin (e.g., `JA[2]`, ball `E17`). Add the pin to `ingest_top.xdc`, re-synthesize, and re-program `ingest_top.bit`.
3. Connect both output signals to a two-channel oscilloscope or logic analyzer. Channel 1: Target A `TX_TRIG` pulse. Channel 2: FPGA `frame_valid` output.
4. The time between the rising edge of Ch. 1 and the rising edge of Ch. 2 is L_{e2e}.

[Figure placeholder]

`LH3D_gpio_latency_waveform.png`

Oscilloscope capture of the GPIO latency measurement. Ch. 1 (Target A `TX_TRIG`, blue) rises at $t = 0$; Ch. 2 (FPGA `frame_valid`, red) rises after L_{e2e}. The cursor measurement shows the end-to-end latency.

Figure 1: Oscilloscope capture of the GPIO latency measurement. Ch. 1 (Target A `TX_TRIG`, blue) rises at $t = 0$; Ch. 2 (FPGA `frame_valid`, red) rises after L_{e2e}. The cursor measurement shows the end-to-end latency.

4.4 Step 4 — Latency measurement at SF7

1. Configure Target A to SF7, +14 dBm, 2-second fixed initiator interval. Power off Target B for this measurement so the only `frame_valid` pulses are from Target A's DATA.
2. Capture at least 20 consecutive latency measurements on the oscilloscope (use the statistics or histogram mode if available).
3. Record: minimum, maximum, mean, and standard deviation of L_{e2e}.
4. Compare the mean to the latency budget: expected ≈50 ms RF on-air time (SF7, 32-byte payload) + 3.73 ms UART + negligible FPGA.
5. Expected: mean close to 55 ms, standard deviation <5 ms (dominated by radio stack jitter, not systematic offsets).

4.5 Step 5 — Latency measurement at SF10

1. Reconfigure Target A to SF10 (update the `g_SPREADING_FACTOR` define in the firmware and reflash). Keep all other parameters the same.
2. Repeat the 20-sample latency measurement.
3. Compare the SF10 mean to the SF7 mean. The difference should be approximately equal to the difference in RF on-air times: SF10 on-air time is ≈370 ms for a 32-byte payload, vs. SF7 ≈50 ms. The measured difference should be close to 320 ms.
4. This comparison validates that the UART and FPGA contributions are small and consistent; the dominant latency component is the RF on-air time.

Timestamp Difference Method (Alternative)

If an oscilloscope is unavailable, a lower-fidelity latency estimate can be obtained from timestamps embedded in the feature vector:

1. The sender-side timestamp (bytes 12–15 of the forwarding frame's embedded radio packet) records the originating Target's STM32 tick at TX time.
2. The interceptor-side timestamp (forwarding frame bytes 2–5) records the Threat's STM32 tick at `RxDone`.
3. The difference approximates $L_{\mathrm{RF}} + L_{\mathrm{RX_FW}}$ (the UART and FPGA contributions are not included).
4. This method has ∼1 ms resolution limited by the STM32 tick period and does not account for the UART transport time. Use it to cross-check the GPIO method, not as the primary measurement.

5 Verification Checklist

- ☐ PER baseline measured for a 5-minute single-station run (Target A only); aggregate value computed.
- ☐ PER decomposition table filled in (RF, UART, CRC, FIFO columns).
- ☐ End-to-end latency measured at SF7 (mean within 15% of theoretical 55 ms).
- ☐ End-to-end latency measured at SF10 (mean within 15% of theoretical 375 ms).
- ☐ SF7 vs. SF10 latency difference is consistent with RF on-air time difference.

Lab Handout 3-D Deliverable

1. PER measurement results: N_{tx}, N_{valid}, aggregate PER, and the four-stage decomposition table.
2. Oscilloscope or logic analyzer captures of the GPIO latency measurement for SF7 and SF10 (at least 20 measurements each). If using oscilloscope statistics, include the histogram screenshot.

3. Summary table: mean, std, min, max latency for SF7 and SF10, alongside the theoretical budget values.
4. Brief discussion (2–4 sentences) identifying which pipeline stage dominates latency and whether the measured values are consistent with the Module 3 latency budget table.

Glossary

FIFO	First In, First Out	2, 3, 5
FPGA	Field-Programmable Gate Array	1, 2, 4, 5
GPIO	General-Purpose Input/Output	4, 5
PER	Packet Error Rate	1, 2, 5
RF	Radio Frequency	1, 2, 3, 4, 5
RTL	Register-Transfer Level	2, 3
UART	Universal Asynchronous Receiver/Transmitter	1, 2, 4, 5
VHDL	VHSIC Hardware Description Language	3